

Testing REST API Servers

Test behavior, not implementation details

Why Test a REST API?

- **Catch regressions:** a change to one handler should not break another endpoint.
- **Confirm the contract:** the method, path, status code, and JSON shape must remain predictable.
- **Find failures early:** invalid input and unsupported routes should fail in a controlled way.
- **Document behavior:** a test shows another developer how the API is expected to respond.

IMPORTANT

| Passing Tests

Passing tests do not prove that an API has no bugs.

- They provide repeatable evidence that the behaviors we selected still work.

Two Levels of API Tests

Level	Question	Example
Connection	Is the server reachable?	<code>GET /</code> returns <code>200</code>
Functional	Does the endpoint behave correctly?	<code>POST /students</code> returns <code>201</code> and the created student

- A connection test is a useful first check, but it does not prove that CRUD operations or validation work.
- For a functional test, verify the complete observable result: **status code + response body + any required follow-up state.**

Tools Used in This Lesson

Tool	Best use
cURL	Quick requests, shell automation, and CI scripts
Fetch	Browser-based checks and frontend integration
Node.js runner	Repeatable JavaScript tests without a browser UI

All three send HTTP requests to the same API. The important part is what the test verifies, not which client sends the request.

cURL Pattern 1: Check a Status Code

Use this when you only need to know whether an endpoint responded with the expected status.

```
status=$(curl -s -o /dev/null -w "%{http_code}" \
http://localhost:8080/students)

if [ "$status" = "200" ]; then
    echo "PASS"
else
    echo "FAIL: expected 200, received $status"
fi
```

- `-s` hides progress output.
- `-o /dev/null` discards the body.
- `-w "%{http_code}"` prints the HTTP status code.

cURL Pattern 2: Send JSON

```
curl -s -X POST \  
  -H "Content-Type: application/json" \  
  -d '{  
    "name": "Test Student",  
    "email": "test@example.edu",  
    "major": "Web Development",  
    "year": 2  
  }' \  
  http://localhost:8080/students
```

The header tells the server how to interpret the request body. A successful create request should return **201 Created** and the new student in **data**.

cURL Pattern 3: Check the Response Body

```
body=$(curl -s http://localhost:8080/students)

if echo "$body" | grep -q '"success":true'; then
    echo "PASS: success is true"
else
    echo "FAIL: unexpected response"
fi
```

`grep -q` reports whether a pattern exists without printing the matching line.

Running the Provided Test Tools

Start the API server:

```
cd module1/code/6_RestAPI/api  
php -S localhost:8080
```

From `module1/code/6_RestAPI`, choose one runner:

```
bash ./api_tests/curl/test_runner_curl.sh --verbose  
node ./api_tests/javascript/test_runner.js --verbose
```

For a visual check, open

`http://localhost:8080/student_api_test.htm`

`l` while the server is running (never open directly as a `file://` file URL).

Install Node.js and npm if necessary:

```
sudo apt update && sudo apt install -y nodejs npm
```

The Same Check with Fetch

```
const response = await fetch('http://localhost:8080/students');
const body = await response.json();

if (response.status !== 200 ||
    body.success !== true ||
    !Array.isArray(body.data)) {
  throw new Error('GET /students failed');
}
```

`fetch()` does not treat HTTP error statuses such as `404` as network errors. Check `response.status` or `response.ok` before accepting the result.

IMPORTANT

Test Structure: Arrange, Act, Assert

1. **Arrange:** prepare input and any data the test needs.
2. **Act:** send one request to the API.
3. **Assert:** compare the observed status and body with the expected result.

```
Arrange: create a valid student payload
Act:      POST /students
Assert:   status is 201
          success is true
          returned fields match the payload
          data.id exists
```

Test 1: GET `/students`

Part	Expected result
Request	<code>GET /students</code>
Status	<code>200 OK</code>
Body	<code>success</code> is <code>true</code>
Body	<code>data</code> is an array
Body	<code>count</code> equals the number of items in <code>data</code>

An empty array is a valid result. A test should not require at least one student unless it created that student during Arrange.

Test 2: GET `/students/{id}`

Part	Expected result
Request	<code>GET /students/{existing-id}</code>
Status	<code>200 OK</code>
Body	<code>success</code> is <code>true</code>
Body	<code>data.id</code> equals the requested ID
Body	Required student fields are present

Also test an ID that does not exist. The expected status is `404 Not Found`, with `success: false` and an error message.

Test 3: POST `/students`

Part	Expected result
Request	Valid JSON object with student fields
Status	<code>201 Created</code>
Body	Returned values match the request
Body	A new numeric <code>id</code> exists
Body	<code>created_at</code> and <code>updated_at</code> exist

Save the returned ID. Use that value for the update and delete tests instead of assuming that a particular ID already exists.

Test 4: PUT `/students/{id}`

Arrange by creating a student first. Then send every required field because this API uses `PUT` as a full replacement.

Part	Expected result
Request	<code>PUT /students/{created-id}</code>
Status	<code>200 OK</code>
Body	Updated fields match the request
Body	<code>id</code> is unchanged
Body	<code>updated_at</code> reflects the update

Test 5: DELETE `/students/{id}`

Part	Expected result
Request	<code>DELETE /students/{created-id}</code>
Status	<code>200 OK</code>
Body	<code>success</code> is <code>true</code>
Body	Deleted student appears in <code>data</code>

Then verify the resulting state:

```
GET /students/{deleted-id} → 404 Not Found
```

The follow-up request proves that the resource is no longer available.

Failure Tests: Routes and Methods

Request	Expected	What it verifies
GET /teachers	404	Unknown resource
GET /students/abc	400	Student ID must be numeric
GET /students/999999	404	Student does not exist
POST /students/1	405	Item path does not allow POST
PUT /students	405	Collection path does not allow PUT

For **405 Method Not Allowed**, also check the **Allow** response header. It should list the methods accepted by that path.

Failure Tests: Request Bodies

All of these requests should return `400 Bad Request`:

Body	Reason
malformed JSON	Body cannot be decoded
<code>[1, 2]</code>	Body is an array, not an object
<code>{}</code>	Required fields are missing
<code>{"name": " ", "email": "bad"}</code>	Field values are invalid
PUT body missing <code>major</code> or <code>year</code>	PUT requires every field

Do not only check for `400`. Confirm that `success` is `false` and that the error message identifies the kind of problem.

IMPORTANT

Keep Tests Independent

- A test should create the data it needs.
- Use the ID returned by POST instead of hard-coding **1**.
- Clean up created records when the test finishes.
- Do not depend on tests running in a particular order unless the suite clearly defines a single CRUD scenario.
- Report enough information to diagnose a failure: test name, expected value, and observed value.

Independent tests can run repeatedly against either the JSON or PDO version of the API.

A Good API Test Is...

| Specific

- Checks one behavior
- Has clear pass/fail criteria

| Reliable

- Produces consistent results
- Controls its own test data

| Complete

- Checks status and body
- Covers success and failure

| Clear

- Names the behavior being tested
- Explains expected and observed results

Common Mistakes

Avoid	Prefer
Checking only <code>success: true</code>	Check the HTTP status first, then the body
Assuming a fixed student ID	Create a record and capture its returned ID
Testing only successful input	Include <code>400</code> , <code>404</code> , and <code>405</code> cases
Copying an entire runner into a slide	Show one request and its essential assertions
Depending on leftover data	Arrange and clean up test data explicitly

IMPORTANT

A Practical Testing Order

1. Start the server and verify `GET /`.
2. Check `GET /students` and its response shape.
3. Create a student and capture the returned ID.
4. Read, update, and delete that student.
5. Verify that the deleted ID returns `404`.
6. Run invalid input, unknown route, and wrong method tests.

For every request, ask the same three questions: